Terraform

Flow of terraform

1. Introduction to IAC
2. Why do we need IAC?
3. Tools of IAC
4. What is terraform
5. History of terraform
6. Features of terraform
7. Architecture of terraform
8. Lifecycle of terraform
9. Terraform hands on practice
   - Installation of terraform
   - Creation of EC2, S3, VPC in terraform
   - Variables in terraform
   - Terraform output
   - Modules in terraform

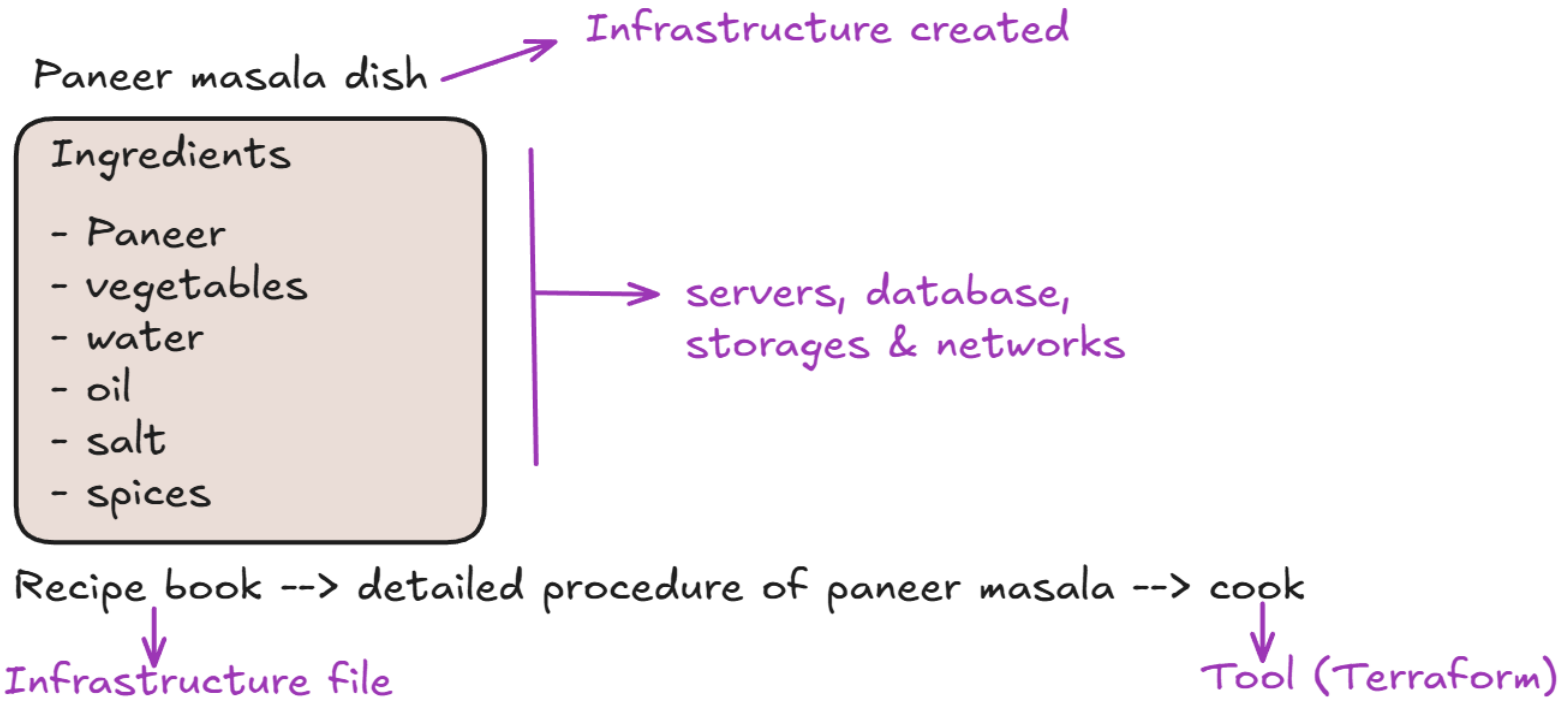
1. Introduction to IAC

IAC - Infrastructure as code

>> It is the process of creating and managing IT infrastructure using the code instead of manually doing it.
Infrastructure such as - Servers : EC2
- Database : RDS
- Networks : VPC
- Storage : S3, EBS

>> Here, the code will be written in order to create the resources(Servers, database, networks, storage) automatically.
>> With infrastructure as code, it enables to represent the infrastructure through machine readable configuration files that supports automation, sharing file across team and version control.

Ex :



2. Why do we need IAC?

1. Scalability - You can easily increase or decrease the resources of infrastructure depending on requirement.
2. Automation - You can easily update or delete any resources or modify the infrastructure file whenever needed using commands.
3. Repeatability - Same infrastructure can be created using the same code.
4. Faster process - Provisioning can happen very quickly.
5. Version control - Tracking the changes made in infrastructure file.
6. Security - Providing the security rules in infrastructure file.
7. Consistency - To ensure same infrastructure file is created every time we run the code due to which we can avoid human errors too.

Tools of IAC

1. Terraform
2. Puppet
3. Pulumi
4. Ansible
5. Bicep
6. AWS CloudFormation

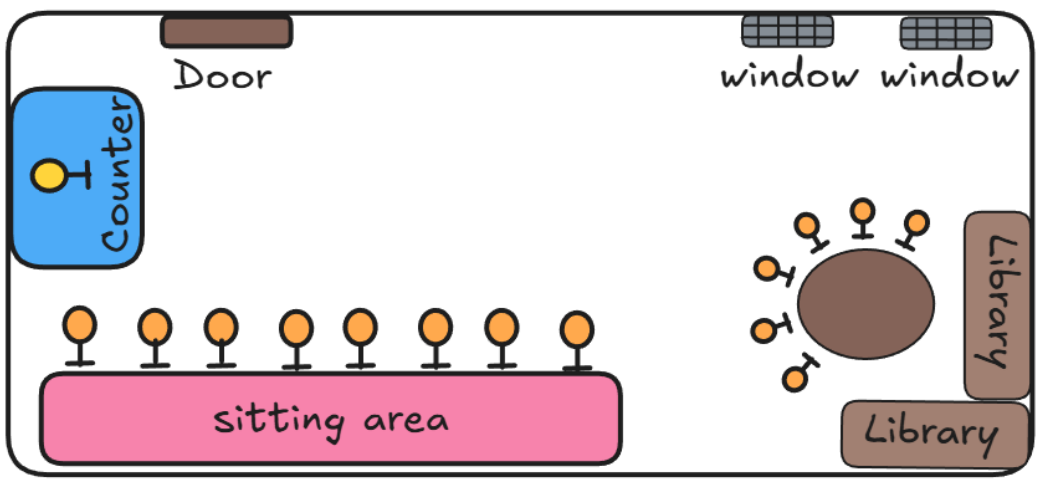
Introduction to terraform

>> Terraform is an open source infrastructure as code tool created by HashiCorp that can be used freely to manage infratructure.

>> Instead of manually performing the required task terraform will help to create the resources like servers, databases, storages and networks automatically.

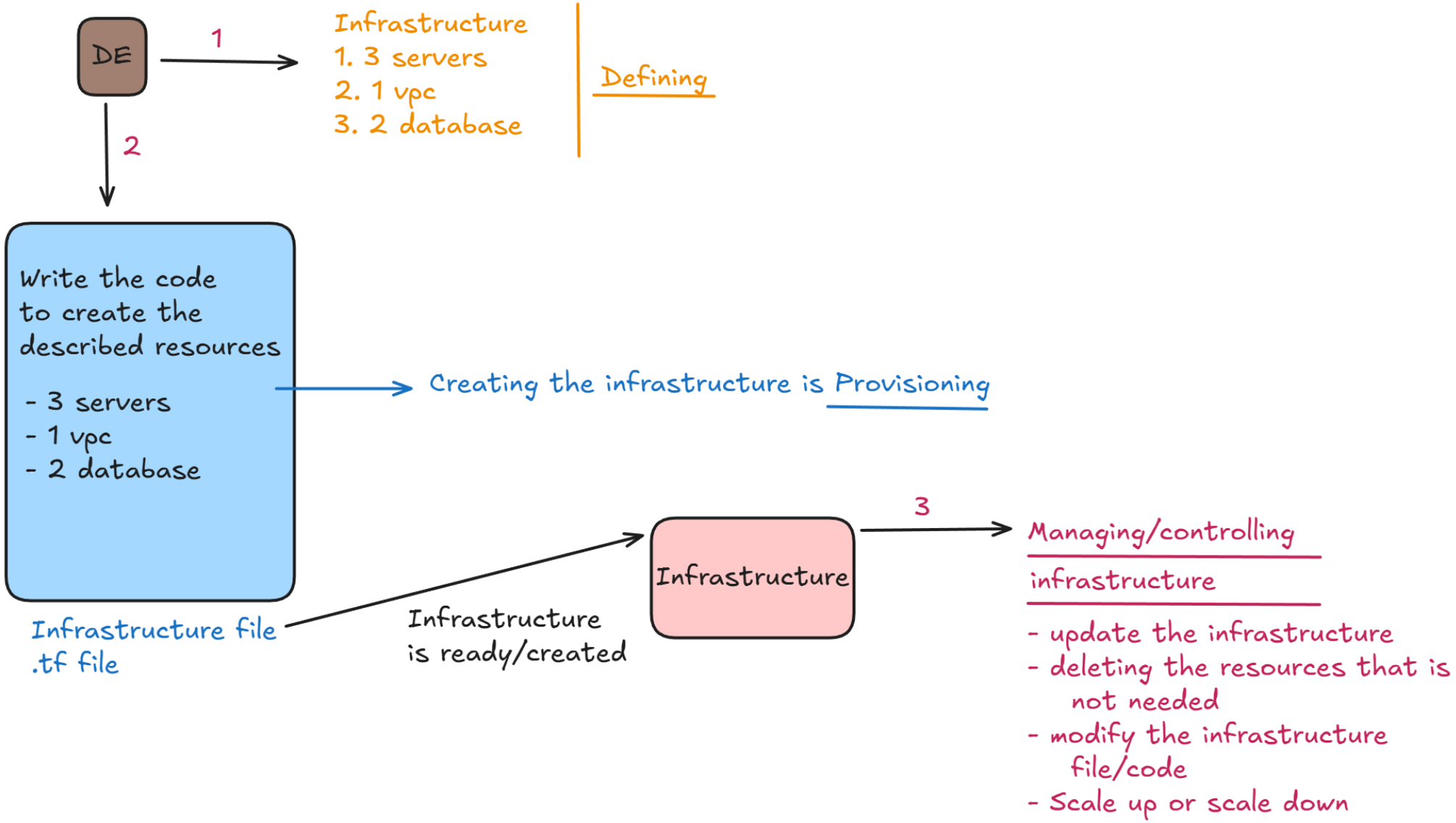
>> Terraform enables to describe, create & control the infrastructures in different cloud platform.
- It lets you define, provision & manage the infrastructure that is been created.

Ex : Cafe coffee



--> The blueprint of cafe

Ex :



History of terraform

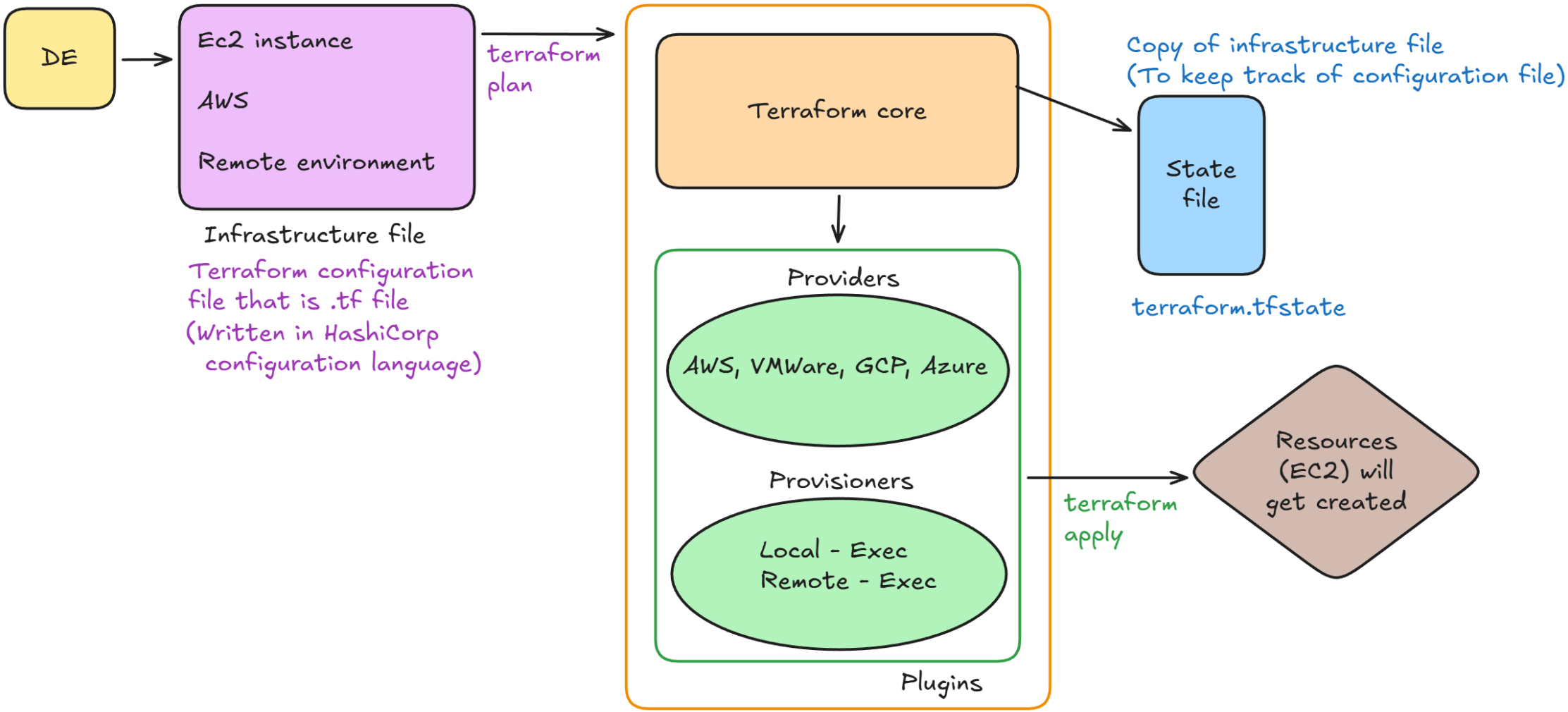
>> Terraform is one of the major tool created by HashiCorp company that was founded by Mitchell Hashimoto and Armon Dadgar in the year 2012.

>> It was officially released in the year 2014 as an open source IAC tool.

Features of terraform

1. It is free and open source
2. Terraform uses declarative approach (JSON format or HCL)
3. It helps tracking & managing the infrastructure state in a file.
4. Terraform shows what will change before execution
5. Terraform works with multiple cloud providers(AWS, GCP, Azure, VMWare.....)

Architecture of terraform



Lifecycle of terraform

>> It defines the sequence of steps that terraform has to follow to create, update, setup configurations, manage the resources and delete the resources if it is not needed.

4 phases of terraform lifecycle.

1. Initialization
2. Planning
3. Apply
4. Destroy

1. Initialization

>> It is the first step in terraform lifecycle where in terraform will prepare the directory by downloading all the necessary components.
- downloading the required providers
- install required plugins
- prepare the terraform to execute any commands smoothly.

command : terraform init

2. Planning

>> It will generate execution preview that shows what terraform intends to change before making the actual execution.

command : terraform plan

3. Apply

>> It is used to execute the changes proposed in the plan & accordingly it will create or update the resources.

command : terraform apply

4. Destroy

>> It is used to delete the resources managed by terraform as described in the infrastructure file.

command : terraform destroy

Hands on execution of EC2, S3, VPC

providers.tf file of EC2

```
# to specify the cloud provider & region to create resources
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "6.27.0"
    }
  }
}

provider "aws" {
  region = "us-west-2"
}
```

main.tf file of EC2

```
# to specify which resource will be created here in aws
resource "aws_instance" "a1instancealias" {
  ami           = "ami-0ebf411a80b6b22cb"  # Amazon linux os and image id
  instance_type = "t3.micro"
  tags = {
    Name = "a1terraforminstance"  # instance name in aws
  }
}

# to create key pair
resource "aws_key_pair" "a1keypairalias" {
  key_name   = "a1terrakeypair"
  public_key = "ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIOYibyWnA+64+g+ZUYkq+Squ/N5wsXhVocoG1qpYV1rO
tanvi@DESKTOP-7EKTOLQ"
}

# to create ebs volume
resource "aws_ebs_volume" "a1ebsalias" {
  availability_zone = "us-west-2b"
  size              = 45
  tags = {
    Name = "a1terravolume"
  }
}

# to attach the volume to instance
resource "aws_volume_attachment" "attachalias" {
  device_name = "/dev/xvdbf"
  volume_id   = aws_ebs_volume.a1ebsalias.id      # resourcename.aliasname.id --> syntax of
                                                                                  volume_id
  instance_id = aws_instance.a1instancealias.id   # resourcename.aliasname.id --> syntax of
                                                                                  instance_id
}
```

providers.tf file of S3

```
# to specify the cloud provider & region to create resources
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "6.27.0"
    }
  }
}

provider "aws" {
  region = "us-west-2"
}
```

main.tf file of S3

```
# to create the bucket
resource "aws_s3_bucket" "s3alias" {
  bucket = "a1terrabucket23456789"
  tags = {
    Name        = "My bucket"
  }
}

# to avoid blocking public access
resource "aws_s3_bucket_public_access_block" "publicaccessalias" {
  bucket = aws_s3_bucket.s3alias.id

  block_public_acls       = false
  block_public_policy     = false
  ignore_public_acls      = false
  restrict_public_buckets = false
}

# to enable the bucket versioning
resource "aws_s3_bucket_versioning" "versioning_alias" {
  bucket = aws_s3_bucket.s3alias.id
  versioning_configuration {
    status = "Enabled"
  }
}

# to add object/files in bucket
resource "aws_s3_object" "objectalias" {
  bucket = "a1terrabucket23456789"
  key    = "terraobject"
  source = "C:/Users/Tanvi/Downloads/newekskey.pem"
}
```

providers.tf file of VPC

```
# to specify the cloud provider & region to create resources
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "6.27.0"
    }
  }
}

provider "aws" {
  region = "us-west-2"
}
```

main.tf file of VPC

```
# to create the vpc
resource "aws_vpc" "vpcalias" {
  cidr_block       = "10.0.0.0/24"
  instance_tenancy = "default"
  tags = {
    Name = "a1vpc"
  }
}

# to create public subnet
resource "aws_subnet" "publicsubnetalias" {
  vpc_id     = aws_vpc.vpcalias.id
  cidr_block = "10.0.0.0/25"
  tags = {
    Name = "publicsubnet"
  }
}

# to create private subnet
resource "aws_subnet" "privatesubnetalias" {
  vpc_id     = aws_vpc.vpcalias.id
  cidr_block = "10.0.0.128/25"
  tags = {
    Name = "privatesubnet"
  }
}
```

```
# to create public route table
resource "aws_route_table" "public_rtalias" {
  vpc_id = aws_vpc.vpcalias.id
  route {
    cidr_block = "10.0.1.0/24"
    gateway_id = aws_internet_gateway.igwalias.id
  }
  tags = {
    Name = "public_rt"
  }
}

# to create private route table
resource "aws_route_table" "private_rtalias" {
  vpc_id = aws_vpc.vpcalias.id

  tags = {
    Name = "private_rt"
  }
}

# to attach public subnet to public route table
resource "aws_route_table_association" "attachpublicalias" {
  subnet_id      = aws_subnet.publicsubnetalias.id
  route_table_id = aws_route_table.public_rtalias.id
}
```

```
# to attach private subnet to private route table
resource "aws_route_table_association" "attachprivatealias" {
  subnet_id      = aws_subnet.privatesubnetalias.id
  route_table_id = aws_route_table.private_rtalias.id
}

# to create internet gateway
resource "aws_internet_gateway" "igwalias" {
  vpc_id = aws_vpc.vpcalias.id

  tags = {
    Name = "a1igw"
  }
}
```

Terraform variables

>> It is used to store values that can be changed & it makes the infrastructure file or terraform code flexible & reusable.

Uses of variables

1. To keep the code secure
2. To change values without touching the main file(main.tf)
3. To reuse same code across different environment
4. To make the code clean.

Types of variables in terraform

1. String - text
2. Map - key value pair
3. Number - numeric value
4. List - n no of values or multiple values
5. Bool - true or false

Syntax :

```
variable "resource_name" {
    description = "detail of what the variable is for"   #optional
    type        = number                                 #optional
    default     = "value"
}
```

To use variable values in main file :

>> var.resource_name

List of variables in EC2, S3 & VPC

EC2

ami_id, key_name, instance_type, instance_name, security_group_ids
user_data, availability_zone, associate_public_ip, root_volume_size, subnet_id

S3

bucket_name, environment, tags, acl, encryption, object_name, enable_versioning, force_destroy

VPC

vpc_name, subnet_name, route_table_name, vpc_cidr, public_subnet_cidrs, private_subnet_cidrs,
availability_zone, enable_internet_gateway, enable_nat_gateway, enable_dns_hostnames

main.tf file

```
# to specify which resource will be created here in aws
resource "aws_instance" "a1instancealias" {
  ami           = "ami-0ebf411a80b6b22cb"  # Amazon linux os and image id
  instance_type = var.instance_type
  tags = {
    Name = "a1terraforminstance"  # instance name in aws
  }
}

# to create key pair
resource "aws_key_pair" "a1keypairalias" {
  key_name   = "a1terrakeypair"
  public_key = "ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIOYibyWnA+64+g+ZUYkq+Squ/N5wsXhVocoG1qpYV1rO
tanvi@DESKTOP-7EKTOLQ"
}

# to create ebs volume
resource "aws_ebs_volume" "a1ebsalias" {
  availability_zone = var.availability_zone
  size              = var.size
  tags = {
    Name = "a1terravolume"
  }
}

# to attach the volume to instance
resource "aws_volume_attachment" "attachalias" {
  device_name = var.device_name
  volume_id   = aws_ebs_volume.a1ebsalias.id      # resourcename.aliasname.id --> syntax of
                                                  volume_id
  instance_id = aws_instance.a1instancealias.id   # resourcename.aliasname.id --> syntax of
                                                  instance_id
}
```

variable.tf file

```
# to add variable for volume az in instance

variable "availability_zone" {
  description = "to add az of volume"
  default = "us-west-2b"
}

# to add the instance type

variable "instance_type" {
  description = "add instance type"
  default = "t3.micro"
}

# to add variable for volume size

variable "size" {
  description = "size of volume"
  default = 40
}

# to add variable for device name

variable "device_name" {
  description = "to add device name"
  default = "/dev/xvdbf"
}
```

Terraform output

>> It is used to display the information about the resources created by terraform after execution.
>> The moment terraform creates resources, output will show the details of the resources.

Uses of terraform output

1. To check important details like IPs, IDs, Hostname & DNS name.
2. To verify the creation of resource
3. Use values in CI/CD pipelines
4. To share the values between modules

Syntax of terraform output

```
output "resource_name" {
    description = "description of output"
    value = "resource_name.alias_name.id"
}
```

How to get terraform output

>> create the output.tf file
>> Write code to get the specific output
>> go for command terraform apply

List of outputs in EC2, S3 & VPC

EC2

instance_id
public_ip
private_ip
public_dns
security_groups
availability_zone
instance_type

S3

bucket_name
bucket_region
bucket_arn
bucket_domain_name
object_name

VPC

vpc_name
vpc_id
igw_id
subnet_name
route_table_ids
route_table_name
vpc_cidr
public_subnet_ids
private_subnet_ids
availability_zone

main.tf file

```
# to specify which resource will be created here in aws
resource "aws_instance" "a1instancealias" {
  ami           = "ami-0ebf411a80b6b22cb"  # Amazon linux os and image id
  instance_type = var.instance_type
  tags = {
    Name = "a1terraforminstance"  # instance name in aws
  }
}

# to create key pair
resource "aws_key_pair" "a1keypairalias" {
  key_name   = "a1terrakeypair"
  public_key = "ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIOYibyWnA+64+g+ZUYkq+Squ/N5wsXhVocoG1qpYV1rO
tanvi@DESKTOP-7EKTOLQ"
}

# to create ebs volume
resource "aws_ebs_volume" "a1ebsalias" {
  availability_zone = var.availability_zone
  size              = var.size
  tags = {
    Name = "a1terravolume"
  }
}

# to attach the volume to instance
resource "aws_volume_attachment" "attachalias" {
  device_name = var.device_name
  volume_id   = aws_ebs_volume.a1ebsalias.id      # resourcename.aliasname.id --> syntax of
                                                                                volume_id
  instance_id = aws_instance.a1instancealias.id   # resourcename.aliasname.id --> syntax of
                                                                                instance_id
}
```

output.tf file

```
# to get the outcome of instance_id

output "instance_id" {
    description = "output of instance_id"
    value = aws_instance.a1instancealias.id
}

# to get the IP

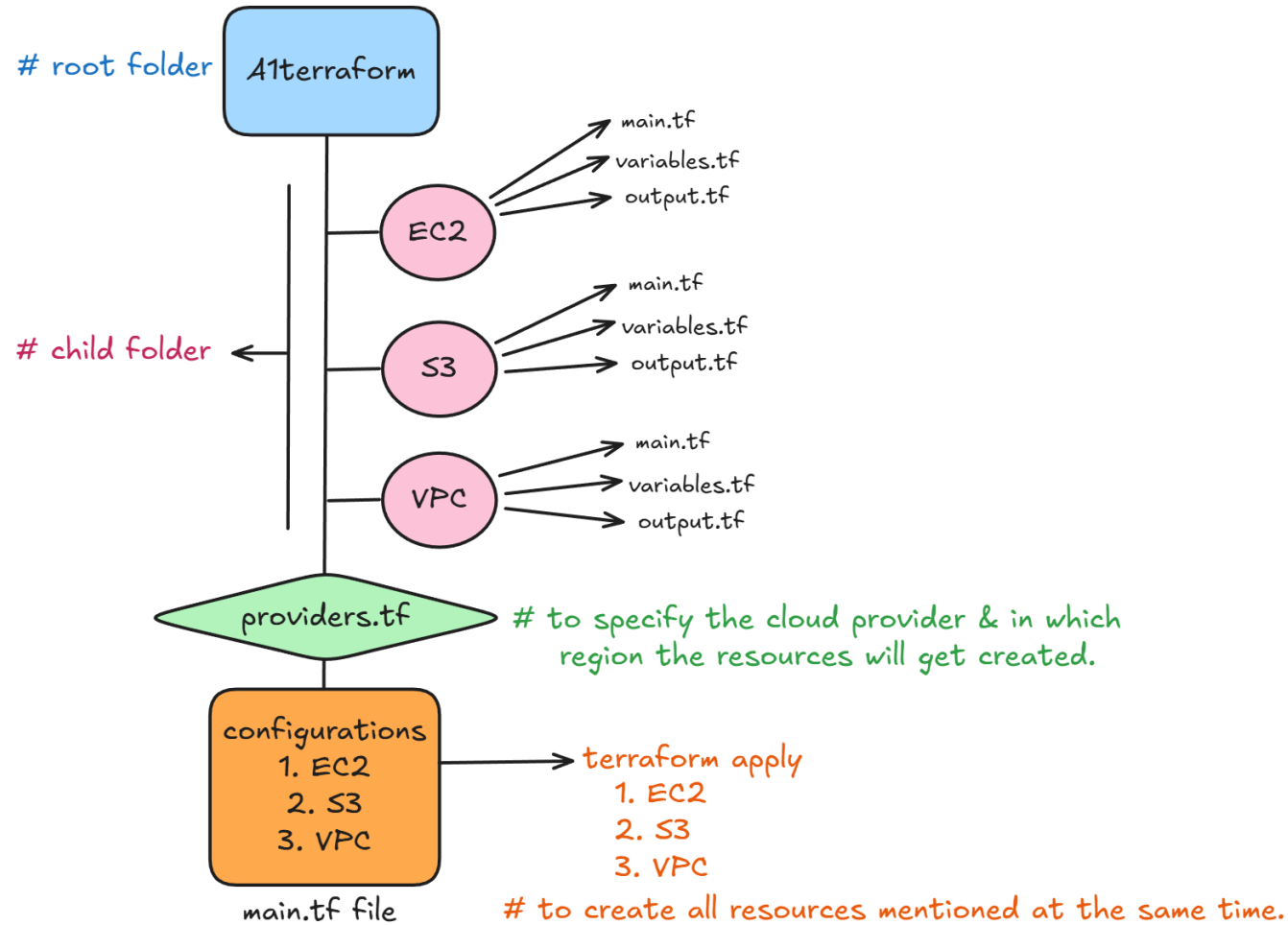
output "instance_public_ip" {
    description = "output of public ip"
    value =
aws_instance.a1instancealias.public_ip
}
```

Terraform modules

>> It is the reusable folder that consist of infrastructure files to create the infrastructures/resources.
>> It consists of terraform code to perform certain tasks.

Uses of terraform modules

1. We can reuse the code many times whenever needed
2. Organized code
3. Clean folder structure
4. Easy to maintain
5. It creates the infrastructures in faster way.



Syntax of module

```
module "module_name_1 {
  source = "module_source_path"
}

module "module_name_2" {
  source = "module_source_path"
}
```

main.tf file of module

```
# to create the resources at same time

module "EC2" {
  source = "./EC2"
}

module "S3" {
  source = "./S3"
}

module "vpc" {
  source = "./vpc"
}
```